

Deep Dive — Hybrid Search, Advanced Patterns, Generation, Evaluation, Failure Modes

Part 08 — Retrieval-Augmented Generation · 08-rag/notes/02-advanced-patterns-and-evaluation.md

This is the practitioner half: what to do when plain vector search isn't good enough (it usually isn't), how to assemble what you retrieved into a trustworthy answer, how to prove any of this actually works, and what breaks in production after the demo ships.

1. Hybrid search: BM25 + dense, fused with Reciprocal Rank Fusion

Dense (embedding) retrieval and sparse (keyword) retrieval fail in complementary ways, and `code/hybrid_search_and_rerank.py` [08-rag/code/hybrid_search_and_rerank.py] demonstrates this with real numbers, not a claim you have to take on faith:

- **BM25** (the sparse, lexical baseline — see the file for the formula implemented from scratch) wins outright on exact-term queries: error codes, part numbers, function names, anything with a literal token that must match. An embedding model has never specifically learned that `ERR_504_TIMEOUT` is a token that matters; it blurs the code into whatever latent topic the surrounding prose belongs to, and loses badly.
- **Dense retrieval** wins on paraphrase and semantic queries that share almost no vocabulary with the target document — exactly what BM25 cannot do, because BM25 only ever sees literal token overlap.
- **Neither wins on both.** This is not a "just use a better embedding model" problem — it's structural. A model that's good at semantics is, almost by construction, willing to treat two different exact tokens as similar if their contexts are similar. That's a feature for paraphrase and a bug for exact match.

Reciprocal Rank Fusion (RRF)

You cannot average a BM25 score and a cosine similarity — they live on different, uncalibrated scales (a BM25 score of 8.3 and a cosine of 0.31 are not comparable numbers, and there's no principled way to rescale one onto the other that survives a change in query). RRF sidesteps calibration entirely by looking only at **rank**, never at score:

```
RRF_score(doc) = sum over each retrieval system of 1 / (k + rank(doc))
```

`k=60` is the value from the original paper (Cormack, Clarke & Buettcher, 2009) and is a fine default — it mainly controls how strongly a system's #1 pick dominates the fusion. Because RRF only consumes ranks, it needs zero score normalization, is robust to one system running "hotter" than the other, and is trivial to extend to three or more retrieval systems (add another rank list, sum the reciprocals).

Reading the numbers from the script: hybrid RRF is never the worst system on either query type — it usually lands within a hair of whichever single system was best for that query, because a document ranked

1 by even one system dominates the fused score.

Opinion worth holding: run hybrid search by default.

The cost (running two retrieval systems) is small compared to the cost of silently losing every exact-term query to a purely dense pipeline, which is the single most common "the search is broken" complaint in production RAG systems that skip BM25 entirely.

2. Reranking: cross-encoder vs. bi-encoder

The embeddings used for retrieval are produced by a **bi-encoder**: the query and each document are encoded *independently* into fixed vectors, and similarity is a single dot product between two vectors computed without ever looking at each other. That's what makes bi-encoders fast enough to index millions of documents — you precompute every document's vector once, offline, and only the query needs encoding at search time.

A **cross-encoder** takes the query and a candidate document *together*, as one input, and outputs a single relevance score. It can attend to fine-grained interactions — exact phrase matches, term coverage, word order — a bi-encoder's single fused vector simply cannot represent. It is dramatically more accurate for ranking, and dramatically more expensive: you cannot precompute a document's score before you know the query, so a cross-encoder cannot be run over a whole corpus — only over a short candidate list.

	Bi-encoder (retrieval)	Cross-encoder (rerank)
Input	query and doc encoded separately	query and doc encoded together
Precomputable	yes — index documents once, offline	no — must run per query at query time
Speed at scale	fast enough for millions of documents	only feasible on a short top-k shortlist
Accuracy	good	noticeably better — sees interactions a bi-encoder discards

The two-stage pattern this implies: retrieve broadly and cheaply (hybrid BM25 + dense, top 20-50), then rerank narrowly and expensively (cross-encoder, top 20-50 → top 5-10 to actually hand the LLM). The script's rerank stage only ever touches the fused top-4 — never the full corpus — which is the whole point. **What reranking cannot do:** recover a relevant document that never made the retrieval shortlist in the first place. It sharpens; it does not substitute for good retrieval. If your eval shows Recall@50 is low, adding a reranker won't fix it — you have a retrieval problem, not a ranking problem.

3. Advanced retrieval patterns — and the specific failure each one targets

Reach for these when hybrid search + rerank still isn't enough. Each solves a *named* problem; don't add one because it sounds sophisticated.

Pattern	Mechanism	Fixes
Query rewriting	An LLM rewrites the user's raw query into one better shaped for retrieval (expands abbreviations, fixes ambiguity, strips conversational filler) before embedding it	Vague or underspecified queries that don't lexically or semantically match the target document
HyDE (Hypothetical Document Embeddings)	Ask the LLM to write a <i>hypothetical answer</i> to the query, then embed and search with that , not the query itself	The query-document asymmetry problem: a short question and a long passage often don't embed near each other even when the passage answers the question perfectly; a hypothetical answer is stylistically closer to the document it's trying to find
Multi-query	Generate several paraphrased versions of the query, retrieve for each independently, then union/fuse the results	A single embedding of a query is one point in space; the right document might be closer to a <i>different phrasing</i> of the same question than to the literal one asked
Parent-document / small-to-big	Index small chunks (precise for search) but retrieve their larger parent chunk or full section to hand to the LLM	The chunk-size trade-off from notes/01 [08-rag/notes/01-core-concepts.md] §2 — precision from a small retrieval unit, completeness from a large generation unit. Sentence-window chunking in code/chunking_strategies.py is this exact pattern at the sentence level
Self-querying	The LLM parses the natural-language query into a semantic part <i>and</i> a structured metadata filter (e.g., "articles about pricing from last month" → embed "pricing" + filter <code>date > last_month</code>)	Queries that mix a semantic ask with a hard structured constraint that vector similarity alone cannot express
Agentic / iterative RAG	The model retrieves, evaluates whether the result is sufficient, and decides whether to retrieve again with a refined query, in a loop — rather than one fixed retrieve-then-generate pass	Multi-hop questions where the answer to sub-question 1 determines what sub-question 2 should even search for; single-pass RAG cannot adapt mid-query. Full treatment of the control loop, planning, and compounding-error math is Module 06 — AI Agents [06-ai-agents]

A caution on HyDE and query rewriting: both add an LLM call *before* retrieval even starts, which adds latency and a second point where hallucination can creep in (a confidently wrong hypothetical document can retrieve confidently wrong real documents). Measure whether they actually move your Recall@k/NDCG@k before keeping them — they help more on ambiguous, conversational queries than on already-well-formed ones, and "it feels smarter" is not evidence.

4. GraphRAG, conceptually

Standard RAG retrieves independent chunks; it has no notion of how entities relate to each other across documents. **GraphRAG** (Microsoft Research, 2024) instead:

1. Runs an LLM over the corpus once, offline, to extract **entities and relationships** (people, orgs, concepts, and how they connect) into a knowledge graph.
2. Runs community detection over that graph to cluster related entities, and has the LLM **summarize each community** — producing a hierarchy of summaries from fine-grained to global.
3. At query time, for questions that need a *global* view across the whole corpus ("what are the main themes across all these documents?") — the kind standard chunk-retrieval RAG answers badly because no single chunk contains "the main themes" — it retrieves from the relevant community summaries instead of individual chunks.

When it's worth the cost: GraphRAG's offline indexing is expensive (multiple LLM passes over the entire corpus to extract entities and generate summaries at each hierarchy level) and the win is concentrated almost entirely on **global, corpus-wide, multi-hop questions** — "how do these five people's projects relate to each other" or "summarize the evolution of this policy across all documents." For the much more common case of "find the passage that answers this specific question," standard chunk retrieval is cheaper and just as good, often better. Don't build a knowledge graph because it sounds more advanced than a vector index; build one because you profiled real user questions and found they're the multi-hop, corpus-spanning kind standard RAG structurally cannot answer.

5. Generation: citation assembly and abstention

Retrieval and reranking only get you a context; the generation step is where trust either gets built or destroyed.

- **Citation assembly:** tag each retrieved chunk with a stable identifier (source doc, section, chunk index) *before* it goes into the prompt, and instruct the model to cite that identifier inline for every claim. This is why structure-aware chunking's header metadata (§4 of `code/chunking_strategies.py`) is valuable beyond retrieval precision — it's what makes "see the Troubleshooting section" possible instead of an opaque, unverifiable chunk ID.
 - **Abstention:** explicitly instruct the model to say "I don't know" or "the provided context doesn't answer this" when retrieval comes back empty, low-confidence, or off-topic — and **test that it actually does**. The default failure mode of an instruction-tuned model handed weak context is to answer anyway, fluently, using its parametric knowledge instead of the (insufficient) retrieved context — which looks identical to a well-grounded answer to a user who can't check. This is the single highest-leverage half-sentence you can add to a RAG prompt, and the one most people forget.
 - **Ordering matters** (see lost-in-the-middle, §7): put the most relevant retrieved chunk first or last in the assembled context, not buried in the middle, and re-verify this after every prompt-template change.
-

6. RAG evaluation metrics

A RAG system fails on two independent surfaces, and one metric can never cover both — you need retrieval metrics and generation metrics, always both. `code/rag_eval_metrics.py` [08-rag/code/rag_eval_metrics.py] implements all of the below from their textbook definitions, with hand-computed self-checks so you can trust the numbers before trusting the pipeline.

Retrieval quality — did we fetch the right evidence?

Metric	Answers	Blind spot
Recall@k	Of all relevant documents, what fraction made the top-k?	Ignores order entirely — a great doc at rank k scores identically to one at rank 1
MRR (Mean Reciprocal Rank)	How far down the list is the <i>first</i> relevant hit?	Ignores everything after the first hit — blind to whether more relevant docs follow
NDCG@k	Rewards relevant docs ranked higher, and higher-grade docs more, all the way through the top-k	Needs graded (not just binary) relevance judgments to be worth computing over plain Recall@k

These three can rank two systems in *opposite* order on the same data — a system that nails one great document at rank 1 wins on MRR; one that surfaces every relevant document, just starting at rank 2, wins on recall. Neither number is wrong; report more than one, and pick the one that matches what your product actually needs (a single-answer QA bot leans on MRR; a synthesize-across-documents task leans on recall).

Generation faithfulness — given that evidence, did the model stick to it? This is the retrieval-adjacent half of what RAGAS (Es et al., 2023) formalizes: decompose the generated answer into claims, and check each against the retrieved context. The real thing uses an LLM/NLI judge per claim; `rag_eval_metrics.py` implements the free, offline proxy version — per-sentence keyword and bigram overlap against context — and is explicit in its own docstring about what that proxy misses (negation, true paraphrase). Use it as a cheap pre-filter to catch egregious fabrication before spending LLM-judge budget on the harder semantic cases. Module 15 — AI Evals [15-ai-evals] covers the full RAGAS metric suite (faithfulness, answer relevance, context precision/recall) run for real with an LLM judge, plus the statistical rigor needed to trust a score computed on a small eval set.

7. Production failure modes

Failure mode	What it looks like	Root cause	Fix
Lost-in-the-middle	The model ignores a correct fact that WAS retrieved, if it sits in the middle of a long context	Transformer attention has a measurable primacy/recency bias — see Module 04 [04-llm] — that degrades for content in the middle of a long prompt	Put the most relevant chunk first or last; retrieve fewer, better chunks rather than many mediocre ones; re-test after every retrieval-count or ordering change
Stale indexes	The system confidently answers with information that was deleted, corrected, or superseded weeks ago	Ingestion is a batch job that ran once; nothing re-embeds or invalidates on document change	Treat the index as a cache with a TTL, not a one-time load; wire re-indexing into your document update path, not a nightly cron you forget exists
Chunk-boundary splits	A number, a name, or a qualifying clause is silently half-present in the retrieved context, producing a subtly wrong (not obviously wrong) answer	Fixed-size or naive chunking cut mid-sentence, mid-clause, or mid-table-row	Boundary-respecting chunking (recursive, sentence-window, structure-aware) — see notes/01 [08-rag/notes/01-core-concepts.md] §2
Retrieval-generation mismatch	Retrieval returns genuinely relevant chunks, but the answer still ignores or contradicts them	The prompt template doesn't force citation/grounding, or the model's instruction-following for "use only the provided context" is weaker than assumed	Explicit abstention instructions (§5), test them adversarially with queries the context does NOT answer, add a faithfulness check to your eval loop, not just a retrieval check
Silent index drift	Retrieval quality degrades slowly over months with no alert, no error, just a rising rate of vague or off-topic answers	Corpus composition changes (new document types, different average length) without anyone re-evaluating the embedding model, chunk size, or ANN recall settings against the new distribution	Recall@k / NDCG@k as a monitored metric, not a one-time launch check — rerun the eval set on a schedule and after every corpus-shape change

The meta-lesson: every failure mode above is invisible in a demo with three well-chosen example queries and shows up only under the query volume and corpus churn of production. Build the eval harness in `code/rag_eval_metrics.py`'s image *before* you need it to diagnose a live incident, not after.

8. Where this returns

Idea here	Where it returns
Lost-in-the-middle, context budget	Module 04 — LLM [04-llm]
Agentic/iterative retrieval, multi-hop planning	Module 06 — AI Agents [06-ai-agents]
Full RAGAS suite, LLM-as-judge, statistical rigor	Module 15 — AI Evals [15-ai-evals]
Retrieval as an agent tool wired into a larger system	Module 07 — Agentic AI Systems [07-agentic-ai]